Week 4 Status Update

1. Planned Work

- Separate the simulator into UI, Core, and Logging processes.
- Implement communication between the independent processes using a suitable IPC mechanism.
- Use threads where appropriate for internal process operations.
- Integrate CPU, Memory, Stack, and FIFO Queue with the Core Process.
- Implement execution and error logging through the Logging Process.
- Integrate the three processes into a complete working simulator.
- Test IPC communication between UI, Core, and Logging processes.
- Compare the standalone simulator with the multi-process simulator.
- Measure execution time, CPU usage, and memory usage.

2. Completed Work

- The simulator was successfully separated into UI, Core, and Logging processes.
- TCP socket communication was implemented as the IPC mechanism.
- UI and Core communication was implemented using TCP port 5001.
- Logging communication was implemented using TCP port 5002.
- LinkedBlockingQueue was used for thread-safe internal communication where required.
- CPU, Memory, Stack, and FIFO Queue operations were integrated with the Core Process.
- PUSH, POP, ENQUEUE, and DEQUEUE operations were tested successfully.
- The Logging Process was integrated and execution information was successfully written to `simulator.log`.
- LOAD, RESET, STEP, RUN, and HALT operations were tested.
- End-to-end communication between the UI, Core, and Logging processes was verified.
- IPC test cases were completed successfully.
- Standalone and multi-process simulator performance measurements were completed.
- Execution time, CPU usage, and memory usage were measured and compared.
- IPC architecture and performance analysis documentation were prepared.

3. Issues Encountered

- Port conflicts occurred when a required server port was already in use.
- Integration of three independent processes required communication and interface testing.

- The distinction between inter-process communication and internal thread communication required clarification during integration.
- RUN and STOP behaviour required coordination because RUN operates through repeated STEP requests.
- Logging required adjustment to avoid unnecessary state-polling messages.
- Performance measurements required separating simulator processes from Java processes belonging to development tools.

4. Decisions Made

- TCP sockets were selected as the IPC mechanism between independent processes.
- Port 5001 was used for communication between the UI and Core processes.
- Port 5002 was used for communication with the Logging Process.
- LinkedBlockingQueue was used only for thread-safe communication within processes and not as the IPC mechanism.
- RUN execution was controlled through repeated STEP requests from the UI.
- Execution information was stored in `simulator.log`.
- The same four-instruction test program was used for standalone and multi-process performance comparison.
- Execution time, CPU usage, and memory usage were selected as the main performance metrics.

5. Individual Contributions

- **Shaima Saleem:** Team Leader – Integration, IPC/interface integration, system testing, and benchmarking.
- **Niranjana S:** Core Process – CPU execution, Memory, Stack, and FIFO Queue.
- **Amra Fathima:** Logging Process – execution and error logging.
- **Thejes Santhosh:** UI Process – user interaction and simulator display.

6. Work Accepted

- Three-process simulator architecture was completed successfully.
- TCP socket-based IPC was implemented and verified.
- UI, Core, and Logging processes were successfully integrated.
- CPU, Memory, Stack, and FIFO Queue functionality was verified after integration.
- Execution logging to `simulator.log` was verified.
- IPC test cases were completed successfully.
- Standalone versus multi-process benchmarking and performance analysis were completed.

7. Work Carried Forward

- Final GitHub documentation update.
- Organization of screenshots and test evidence.

- Final demonstration preparation.

8. Week 5 Plan

- Complete and organize the final project documentation.
- Verify the complete simulator before the final demonstration.
- Organize test results, benchmark results, and screenshots.
- Prepare the team for the final project presentation and demonstration.